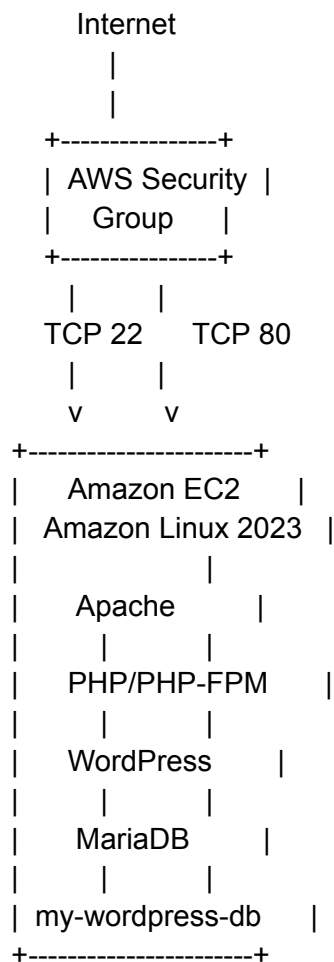


Deploy WordPress on Amazon EC2

A hands-on AWS project demonstrating how to deploy a WordPress website on an Amazon EC2 instance running **Amazon Linux 2023**.

The project covers EC2 provisioning, Security Groups, Linux administration, Apache, PHP/PHP-FPM, MariaDB, WordPress configuration, database management, file permissions, and Apache permalink configuration.

Architecture



Project Overview

In this project, I deployed a WordPress website on an Amazon EC2 instance using the AWS Management Console and SSH.

The EC2 instance runs **Amazon Linux 2023** and hosts:

- Apache HTTP Server
- PHP
- PHP-FPM
- MariaDB
- WordPress
- phpMyAdmin (optional)

I also configured Linux file permissions, created a dedicated database and database user, configured WordPress, and enabled Apache `.htaccess` support for WordPress permalinks.

AWS Services

- Amazon EC2
 - Amazon VPC
 - Security Groups
 - Public IPv4
 - AWS Management Console
-

Technologies

- Amazon Linux 2023
 - Apache HTTP Server
 - PHP
 - PHP-FPM
 - MariaDB
 - WordPress
 - phpMyAdmin
 - SSH
 - Bash
 - SQL
-

Lab Objectives

By completing this project, I practiced:

- Launching an EC2 instance
 - Configuring an EC2 Security Group
 - Connecting to Linux using SSH
 - Managing Linux packages with `dnf`
 - Installing and configuring Apache
 - Installing PHP and PHP-FPM
 - Installing and securing MariaDB
 - Creating a database and database user
 - Configuring WordPress
 - Managing Linux file permissions
 - Configuring Apache `.htaccess`
 - Enabling WordPress permalinks
 - Troubleshooting AWS/Linux services
-

1. Launch EC2 Instance

AWS Region

US East (N. Virginia)
us-east-1

Instance Configuration

Setting	Value
AMI	Amazon Linux 2023
Architecture	x86_64
Instance Type	t2.micro / t3.micro
Storage	Default
Name	WordPressInstance

This project uses **Amazon Linux 2023**, not the older Amazon Linux 2 used in the original training lab.

2. Configure Security Group

The EC2 Security Group was configured with the following inbound rules:

Type	Protocol	Port	Source
SSH	TCP	22	My IP
HTTP	TCP	80	0.0.0.0/0
HTTPS	TCP	443	Optional

Security Consideration

SSH should preferably be restricted to my current public IP address rather than allowing:
0.0.0.0/0

HTTP must be publicly accessible if the WordPress website needs to be accessed from the Internet.

HTTPS can be added later when TLS/SSL is configured.

3. Connect to EC2 Using SSH

Set the private key permissions:

```
chmod 400 WordPressLabKey.pem
```

Connect to the EC2 instance:

```
ssh -i WordPressLabKey.pem ec2-user@PUBLIC-IP
```

Replace:

PUBLIC-IP

with the public IPv4 address of the EC2 instance.

4. Verify the Operating System

Because Amazon Linux 2023 is being used, verify the OS:

```
cat /etc/os-release
```

Expected output includes:

```
NAME="Amazon Linux"  
VERSION="2023"  
ID="amzn"  
PRETTY_NAME="Amazon Linux 2023..."
```

Check the architecture:

```
uname -m
```

Expected:

```
x86_64
```

5. Update the System

Amazon Linux 2023 uses `dnf`.

```
sudo dnf upgrade -y
```

The original training lab uses `yum` and `amazon-linux-extras`, which are associated with older Amazon Linux workflows. Amazon Linux 2023 uses `dnf`, and `amazon-linux-extras` is not available.

6. Install Apache, PHP and PHP-FPM

Install Apache and PHP components:

```
sudo dnf install -y httpd wget php-fpm php-mysql php-json php php-devel
```

Start Apache:

```
sudo systemctl start httpd
```

Enable Apache at boot:

```
sudo systemctl enable httpd
```

Or enable and start it in one command:

```
sudo systemctl enable --now httpd
```

Check Apache:

```
sudo systemctl status httpd
```

Verify it is enabled:

```
sudo systemctl is-enabled httpd
```

Expected:

```
enabled
```

7. Start PHP-FPM

Start PHP-FPM:

```
sudo systemctl start php-fpm
```

Enable PHP-FPM at boot:

```
sudo systemctl enable php-fpm
```

Or:

```
sudo systemctl enable --now php-fpm
```

Check the service:

```
sudo systemctl status php-fpm
```

Check PHP version:

```
php -v
```

8. Test Apache

Open the following URL in a browser:

```
http://PUBLIC-IP
```

If the Apache test page appears, the web server is running correctly.

You can also verify that Apache is listening on port 80:

```
sudo ss -ltnp | grep ':80'
```

9. Configure Linux Permissions

Add `ec2-user` to the Apache group:

```
sudo usermod -a -G apache ec2-user
```

Log out:

```
exit
```

SSH back into the instance.

Verify group membership:

```
groups
```

Change ownership:

```
sudo chown -R ec2-user:apache /var/www
```

Set directory permissions:

```
sudo chmod 2775 /var/www
```

Apply the same permissions to subdirectories:

```
find /var/www -type d -exec sudo chmod 2775 {} \;
```

Apply file permissions:

```
find /var/www -type f -exec sudo chmod 0664 {} \;
```

Why?

These permissions allow the EC2 user and Apache group to work with files under:

`/var/www`

These permissions are suitable for this hands-on lab. Production environments should use stricter least-privilege permissions.

10. Test PHP

Create a temporary PHP information page:

```
echo '<?php phpinfo(); ?>' | sudo tee /var/www/html/phpinfo.php
```

Open:

```
http://PUBLIC-IP/phpinfo.php
```

If the PHP information page appears, PHP is working.

Important Security Step

Remove the test file after verification:

```
sudo rm /var/www/html/phpinfo.php
```

`phpinfo()` can expose sensitive configuration information and should not be publicly available on a production server.

11. Install MariaDB

Install MariaDB:

```
sudo dnf install -y mariadb105-server
```

Start MariaDB:

```
sudo systemctl start mariadb
```

Enable MariaDB at boot:

```
sudo systemctl enable mariadb
```

Or:

```
sudo systemctl enable --now mariadb
```

Check the service:

```
sudo systemctl status mariadb
```

Check the MariaDB version:

```
sudo mariadb
```

Then:

```
SELECT VERSION();
```

Exit:

```
exit;
```

12. Secure MariaDB

Run:

```
sudo mariadb-secure-installation
```

If that command is not available, try:

```
sudo mysql_secure_installation
```

Follow the prompts to:

- Set or configure the database root authentication
- Remove anonymous users
- Disable remote root login
- Remove the test database
- Reload privilege tables

This reduces unnecessary database exposure.

13. Optional: Install phpMyAdmin

phpMyAdmin is a web-based database administration tool.

Install required PHP extensions:

```
sudo dnf install -y php-mbstring php-xml
```

Restart PHP-FPM:

```
sudo systemctl restart php-fpm
```

Restart Apache:

```
sudo systemctl restart httpd
```

Go to the Apache document root:

```
cd /var/www/html
```

Download the latest phpMyAdmin package:

```
wget https://www.phpmyadmin.net/downloads/phpMyAdmin-latest-all-languages.tar.gz
```

Create the phpMyAdmin directory and extract the files:

```
mkdir -p phpMyAdmin
```

```
tar -xvzf phpMyAdmin-latest-all-languages.tar.gz -C phpMyAdmin --strip-components 1
```

Remove the downloaded archive:

```
rm phpMyAdmin-latest-all-languages.tar.gz
```

Make sure MariaDB is running:

```
sudo systemctl start mariadb
```

Open:

```
http://PUBLIC-IP/phpMyAdmin
```

Security Note

phpMyAdmin should not be left publicly exposed without additional security controls in a production environment.

For production, consider:

- HTTPS
- IP restrictions
- VPN or private access
- Strong authentication
- WAF
- Removing phpMyAdmin when it is no longer needed

14. Download WordPress

Move to `/var/www`:

```
cd /var/www
```

Download WordPress:

```
wget https://wordpress.org/latest.tar.gz
```

Extract the archive:

```
tar -xzf latest.tar.gz
```

This creates:

```
/var/www/wordpress
```

15. Create the WordPress Database

Open MariaDB:

```
sudo mariadb
```

Create the database:

```
CREATE DATABASE `my-wordpress-db`;
```

Create a dedicated WordPress database user:

```
CREATE USER 'wordpress_user'@'localhost' IDENTIFIED BY  
'YOUR_STRONG_PASSWORD';
```

Grant permissions only to the WordPress database:

```
GRANT ALL PRIVILEGES ON `my-wordpress-db`.* TO 'wordpress_user'@'localhost';
```

Apply the changes:

```
FLUSH PRIVILEGES;
```

Verify the database:

```
SHOW DATABASES;
```

Verify the user's privileges:

```
SHOW GRANTS FOR 'wordpress_user'@'localhost';
```

Exit:

```
exit;
```

Security Principle

WordPress does not need to connect to MariaDB using the database root account.

Instead, a dedicated database user is created:

```
wordpress_user
```

This follows the **least privilege principle**.

16. Configure WordPress

Go to:

```
cd /var/www
```

Copy the sample configuration:

```
cp wordpress/wp-config-sample.php wordpress/wp-config.php
```

Open the configuration file:

```
nano wordpress/wp-config.php
```

Update these values:

```
define( 'DB_NAME', 'my-wordpress-db' );  
define( 'DB_USER', 'wordpress_user' );  
define( 'DB_PASSWORD', 'YOUR_STRONG_PASSWORD' );  
define( 'DB_HOST', 'localhost' );
```

Do not use the real password in a GitHub README.

17. Configure WordPress Security Keys

WordPress uses authentication keys and salts to improve the security of user authentication cookies.

Generate fresh values using the official WordPress salt generator:

<https://api.wordpress.org/secret-key/1.1/salt/>

Replace the sample key and salt values in:

`wp-config.php`

with the generated values.

Important

Never publish real:

- Database passwords
- WordPress passwords
- AWS credentials
- Private keys
- API keys
- Access tokens
- Secrets

in GitHub.

18. Deploy WordPress to Apache Document Root

Create the website directory:

```
sudo mkdir -p /var/www/html/mywordpresswebsite
```

Copy WordPress files:

```
sudo cp -r wordpress/* /var/www/html/mywordpresswebsite/
```

Verify:

```
ls -la /var/www/html/mywordpresswebsite
```

19. Configure Apache for WordPress Permalinks

WordPress uses `.htaccess` for features such as pretty permalinks.

Open Apache configuration:

```
sudo nano /etc/httpd/conf/httpd.conf
```

Find:

```
<Directory "/var/www/html">
```

Inside this specific section, find:

```
AllowOverride None
```

Change it to:

```
AllowOverride All
```

Important

There may be multiple `AllowOverride` directives in the Apache configuration.

Change the one inside:

<Directory "/var/www/html">

20. Validate Apache Configuration

Before restarting Apache:

```
sudo apachectl configtest
```

Expected:

Syntax OK

Restart Apache:

```
sudo systemctl restart httpd
```

21. Verify File Permissions

Make sure Apache can access the WordPress files:

```
sudo chown -R ec2-user:apache /var/www
```

Set directory permissions:

```
sudo chmod 2775 /var/www
```

```
find /var/www -type d -exec sudo chmod 2775 {} \;
```

Set file permissions:

```
find /var/www -type f -exec sudo chmod 0664 {} \;
```

Restart Apache:

```
sudo systemctl restart httpd
```

22. Enable Services at Boot

Enable Apache:

```
sudo systemctl enable httpd
```

Enable PHP-FPM:

```
sudo systemctl enable php-fpm
```

Enable MariaDB:

```
sudo systemctl enable mariadb
```

Check all services:

```
sudo systemctl status httpd
```

```
sudo systemctl status php-fpm
```

```
sudo systemctl status mariadb
```

23. Run the WordPress Installation

Open the following URL:

<http://PUBLIC-IP/mywordpresswebsite>

The WordPress installation page should appear.

Enter:

- Site Title

- WordPress Administrator Username
- Strong Password
- Email Address

Then complete the installation.

24. Log In to WordPress

After installation, open:

`http://PUBLIC-IP/mywordpresswebsite/wp-admin`

Log in using the WordPress administrator account created during installation.

The WordPress administrator account is different from the Linux `ec2-user` account and the MariaDB database user.

25. Validation

The following components should now be working:

EC2

- Amazon Linux 2023
- Security Group
- SSH
- Apache
- PHP
- PHP-FPM
- MariaDB
- WordPress

Validation Commands

Check Apache:

`sudo systemctl status httpd`

Check PHP:

```
php -v
```

Check PHP-FPM:

```
sudo systemctl status php-fpm
```

Check MariaDB:

```
sudo systemctl status mariadb
```

Check Apache configuration:

```
sudo apachectl configtest
```

Check port 80:

```
sudo ss -ltnp | grep ':80'
```

Check WordPress files:

```
ls -la /var/www/html/mywordpresswebsite
```

Conclusion

This project demonstrates a complete hands-on deployment of WordPress on AWS using Amazon EC2.

The lab combines **AWS cloud infrastructure, Linux administration, web server configuration, database management, application deployment, and security best practices** in a single practical project.